

**LAPORAN PRAKTIKUM
STRUKTUR DATA
“ABSTRACT DATA TYPE (ADT)”
PADA BAHASA PEMROGRAMAN JAVA**

disusun Oleh :

RAZIF AL FARISI

2511532028

Dosen Pengampu : Dr. Wahyudi S.T, M.T

Asisten Laboratorium : Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul “**Abstract Data Type (ADT) pada Bahasa Pemrograman Java**” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum pada mata kuliah Struktur Data, khususnya mengenai konsep Abstract Data Type (ADT). Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep ADT serta implementasinya dalam bahasa pemrograman Java.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data yang telah memberikan ilmu dan arahan selama proses pembelajaran.

Padang, 30 November 2025

Penulis

Daftar Isi

KATA PENGANTAR	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
BAB II	4
PEMBAHASAN	4
2.1 Landasan Teori	4
2.2 Langkah Pengerjaan.....	Error! Bookmark not defined.
2.2.1 Setting IP Address.....	Error! Bookmark not defined.
2.2.2 File Sharing	Error! Bookmark not defined.
BAB III	9
PENUTUP.....	9
3.1 Kesimpulan.....	9
3.2 Saran.....	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Abstract Data Type (ADT) merupakan konsep dalam struktur data yang mendefinisikan tipe data berdasarkan perilaku (operasi) yang dapat dilakukan terhadap data tersebut, tanpa memperhatikan bagaimana data tersebut diimplementasikan secara detail. *ADT* membantu dalam pengelolaan data secara lebih terstruktur dan terorganisir.

Dalam pemrograman Java, ADT sering diimplementasikan dalam bentuk class yang memiliki atribut dan method. Dengan menggunakan ADT, programmer dapat memisahkan antara representasi data dan operasi yang dilakukan terhadap data tersebut., Salah satu contoh implementasi ADT adalah tipe data waktu (jam) yang terdiri dari jam, menit, dan detik. Dengan ADT, kita dapat melakukan berbagai operasi seperti konversi waktu, penjumlahan waktu, serta perbandingan waktu.

1.2 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah sebagai berikut:

1. Memahami konsep *Abstract Data Type (ADT)*.
2. Mengimplementasikan ADT dalam bahasa Java.
3. Memahami penggunaan method seperti konstruktor, selektor, dan mutator.
4. mengimplementasikan operasi pada ADT seperti konversi, relasional, dan aritmatika.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Abstract Data Type (ADT) adalah suatu model data yang mendefinisikan tipe data melalui operasi-operasi yang dapat dilakukan terhadapnya tanpa memperhatikan detail implementasinya.

Dalam Java, ADT direpresentasikan menggunakan class yang terdiri dari :

- Atribut (data)
- *Method* (operasi)

Beberapa jenis method dalam *ADT* :

- Konstruktor : Membuat objek
- Selektor (*getter*) : Mengambil nilai
- Mutator (*setter*) : Mengubah nilai
- *Method* operasi : Manipulasi data

2.2 Implementasi Program

Berikut adalah Implementasi ADT Jam :

```
package pekan1_2511532028;

public class Jam_2511532028 {
    private int hh;
    private int mm;
    private int ss;

    // Validator
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && ( 0 <= m && m <= 59) && ( 0 <= s && s
<= 59);
    }

    // konstruktor
    public Jam_2511532028(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    // Selektor
    public int getHour() { return hh;}
    public int getMinute() { return mm;}
    public int getSecond() { return ss;}

    // Mutator
    public void setHour(int h) {this.hh = h;}
```

```

    public void setMinute(int m) { this.mm = m;}
    public void setSecond(int s) { this.ss = s;}

    //konversi
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511532028 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600;
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511532028(h, m, s);
    }

    // Relasional
    public int compareTo(Jam_2511532028 other) {return
Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if(! (o instanceof Jam_2511532028 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() { return java.util.Objects.hash(hh, mm, ss); }

    // Aritmatika
    public Jam_2511532028 plus(Jam_2511532028 other) { return
fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511532028 minus(Jam_2511532028 other) { return
fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
    public Jam_2511532028 nextSecond() { return fromSeconds(this.toSeconds() +
1); }
    public Jam_2511532028 nextNSecond( int n) { return
fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511532028 prevSecond() { return
fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
    public Jam_2511532028 prevNSecond( int n) { return
fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }

    // Durasi (detik)
    public static int durasiDetik(Jam_2511532028 jaw, Jam_2511532028 jakh) {
return jakh.toSeconds() - jaw.toSeconds(); }

    public String toString() { return String.format("%02d:%02d:%02d", hh, mm,
ss); }
}

```

Untuk class main 1 (JamDriver) :

```

package pekan1_2511532028;

public class JamDriver_2511532028 {
    public static void main(String[] args) {
        Jam_2511532028 a = new Jam_2511532028(23 , 59, 50);
        Jam_2511532028 b = new Jam_2511532028(0, 0 , 15);
        System.out.println("a          = " + a);
        System.out.println("b          = " + b);
        System.out.println("a+b      = " + a.plus(b));
    }
}

```

```

        System.out.println("next 20s" + " +
a.nextNSecond(20));
        System.out.println("durasi (a,b) = " +
Jam_2511532028.durasiDetik(a,b));
        System.out.println("a.compareTo(b) = " +
a.compareTo(b));
    }
}

```

Untuk class main 2 (JamDriver2) :

```

package pekan1_2511532028;

import java.util.Scanner;

public class JamDriver2_2511532028 {

    public static void main(String[] args) {
        Scanner i = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. input jam pertama
        System.out.println("\n--- Input Jam ---");
        Jam_2511532028 j1 = buatJamDariInput(i);

        // w. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511532028 j2 = buatJamDariInput(i);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String) : " +
j1.toString());
        System.out.println("Jam 2 (String) : " +
j2.toString());
        System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik : " + j2.toSeconds());

        // 4. Operasi Relasional (perbandingan)
        int perbandingan = j1.compareTo(j2);
        if(perbandingan > 0) {
            System.out.println("Status : Jam 1
lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0 ) {
            System.out.println("Status : Jam 1
lebih awal (setelah) Jam 2");
        } else {
            System.out.println("Status : Jam 1
dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2) : " +
Jam_2511532028.durasiDetik(j1, j2) + " detik" );

        Jam_2511532028 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya : " + jNext);
    }
}

```

```

        Jam_2511532028 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);

        //6. Operasi Penjumlahan Jam
        Jam_2511532028 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 = J2 : " +
jHasilPlus);

        i.close();
        System.out.println("\nProgram Selesai.");
    }

    private static Jam_2511532028 buatJamDariInput(Scanner sc) {
        int h, m, s;
        while (true) {
            System.out.print("Masukkan Jam (0-23) : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // memanggil method isValid
            if (Jam_2511532028.isValid(h, m, s)) {
                return new Jam_2511532028(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan
ulangi.\n");
            }
        }
    }
}

```

2.3 Penjelasan Program

A. ADT Jam

Program di atas merupakan implementasi ADT Jam yang memiliki atribut jam (hh), menit (mm), dan detik (ss).

- Validator (isValid) digunakan untuk memastikan input waktu valid
- Konstruktor digunakan untuk membuat objek waktu
- Selektor digunakan untuk mengambil nilai waktu
- Mutator digunakan untuk mengubah nilai waktu

Selain itu, terdapat beberapa operasi:

- Konversi : mengubah waktu ke detik dan sebaliknya
- Relasional : membandingkan dua waktu
- Aritmatika : penjumlahan dan pengurangan waktu
- Navigasi waktu : nextSecond, prevSecond

Program ini menunjukkan bahwa ADT dapat mengelola data secara terstruktur dan menyediakan berbagai operasi yang berkaitan dengan data tersebut.

B. JamDriver1

Program ini menggunakan data langsung.

1. Membuat objek :

Jam a = 23:59:50

Jam b = 00:00:15

2. Menampilkan :

- Nilai jam
- Hasil penjumlahan jam (**a + b**)
- Penambahan detik (**nextNSecond(20)**)
- Durasi
- Perbandingan (**compareTo**)

C. JamDriver2

Program ini menggunakan inputan (keyboard) dari user.

1. Input data

- Jam
- Menit
- Detik

2. Validasi

'Jam_2511532028.isValid(h, m, s)'

Memastikan input sesuai range.

3. Konversi

j1.toSeconds()

Mengubah jam ke detik.

4. Perbandingan

j1.compareTo(j2)

5. Operasi yang dilakukan

- toSecond() : konversi ke detik
- compareTo() : membandingkan waktu
- durasiDetik() : selisih waktu
- nextSecond() : tambah 1 detik
- prevSecond() : kurang 1 detik
- plus() : penjumlahan waktu

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa *Abstract Data Type (ADT)* merupakan konsep penting dalam struktur data yang memungkinkan pengelolaan data secara terstruktur dan terorganisir.

Implementasi ADT dalam Java dapat dilakukan menggunakan class yang berisi atribut dan method. Dengan adanya konstruktor, selektor, mutator, serta operasi lainnya, *ADT* dapat digunakan untuk memodelkan suatu objek dengan baik.

3.2 Saran

Dalam mempelajari ADT, sebaiknya mahasiswa memahami konsep dasar terlebih dahulu sebelum mengimplementasikannya dalam kode program. Selain itu, latihan membuat berbagai ADT akan membantu meningkatkan pemahaman terhadap struktur data

Referensi

Data-Structures-and-Algorithms-in-Java-6th-Edition.pdf

https://www.academia.edu/18364459/Diktat_Sturktur_Data